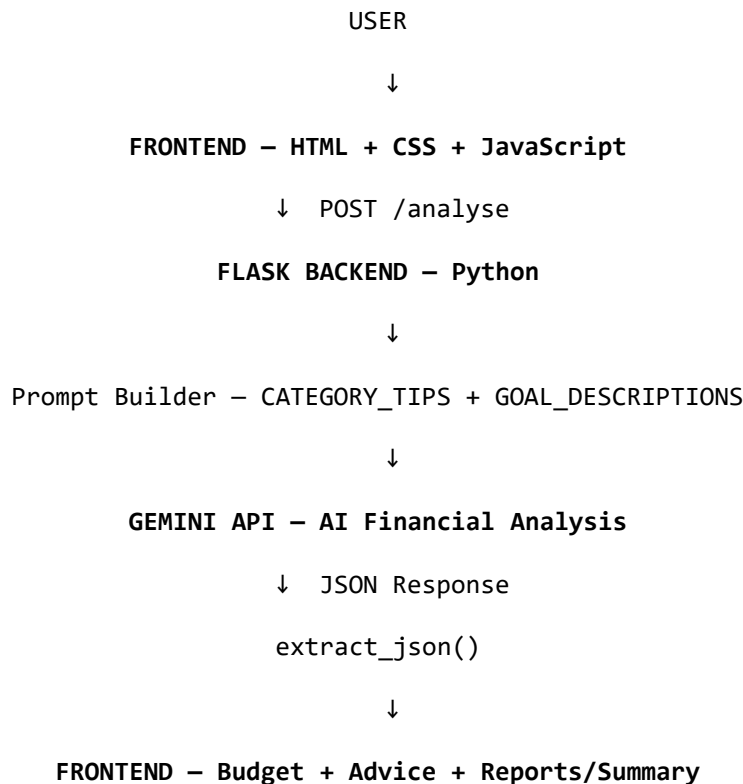


Personal Finance Advisor Bot

Application Architecture and Technical Design

1. Draft an Architectural Diagram

The Personal Finance Advisor Bot follows a simple frontend–backend–AI architecture. The user interacts with the frontend, the Flask backend processes the request, and the Gemini API generates personalized financial analysis.



2. Frontend Functionality

The frontend provides a simple interface through which users enter their financial information and view the AI-generated results.

User input fields:

- Monthly income
- Expense categories such as food, transport, shopping, entertainment, and rent
- Financial goal, such as saving for a laptop or travel
- Savings amount or savings target
- Investment preference or risk level, if the investment module is included

Example user input:

- Monthly Income: ₹30,000
- Food: ₹5,000
- Transport: ₹2,000
- Shopping: ₹3,000
- Entertainment: ₹2,000
- Rent: ₹8,000
- Goal: Save for a laptop

After clicking “Analyse My Finances”, JavaScript sends the information to the Flask backend using a POST request to /analyse.

Example results displayed to the user:

- Monthly Income: ₹30,000
- Total Expenses: ₹20,000
- Potential Savings: ₹10,000
- AI suggestions such as reducing unnecessary shopping expenses and increasing monthly savings

3. Backend Responsibilities

The Flask backend acts as the main processing layer between the frontend and Gemini API.

Receive Request: Flask receives the user's financial data through the POST /analyse endpoint.

Validate Input: The backend checks that values such as income and expenses are valid numbers and that invalid or negative values are rejected.

Sanitise Input: The backend cleans user input before placing it into the AI prompt.

Build Financial Prompt: Flask combines the user's income, expenses, goals, and predefined guidance into a structured prompt.

Call Gemini API: The backend sends the structured prompt to the Gemini API for analysis.

Process Response: The returned AI response is processed using `extract_json()` to obtain structured JSON.

Return Result: Flask returns the structured result to the frontend so JavaScript can display it.

4. AI Integration Points

The AI integration is responsible for turning the user's financial information into personalized budget recommendations, spending analysis, saving suggestions, and monthly summaries.

Overall AI flow:

1. User financial data
2. Flask backend
3. CATEGORY_TIPS and GOAL_DESCRIPTIONS
4. Structured financial prompt

5. Gemini API
6. AI-generated JSON response
7. `extract_json()`
8. Structured result returned to frontend

CATEGORY_TIPS Dictionary

CATEGORY_TIPS contains predefined guidance for common expense categories.

```
CATEGORY_TIPS = {  
    "food": "Look for ways to reduce unnecessary food spending.",  
    "shopping": "Avoid unnecessary purchases and compare prices.",  
    "transport": "Consider public transport or shared travel.",  
    "entertainment": "Set a monthly limit for entertainment."  
}
```

GOAL_DESCRIPTIONS Dictionary

GOAL_DESCRIPTIONS provides descriptions of the user's financial goals so that the AI can tailor its advice.

```
GOAL_DESCRIPTIONS = {  
    "saving": "Help the user increase monthly savings.",  
    "laptop": "Help the user save enough money to purchase a laptop.",  
    "travel": "Help the user plan savings for a future trip."  
}
```

extract_json() Function

Gemini may return additional text around the requested data. The `extract_json()` function is used to identify and extract the JSON portion of the response so that Flask can return clean, structured data to the frontend.

Example structured response:

```
{  
    "budget": {  
        "food": 4000,  
        "transport": 2000,  
        "shopping": 2000  
    },  
    "savings": 10000,  
    "suggestions": [  
        "Reduce shopping expenses",  
        "Increase monthly savings"  
    ]  
}
```

5. Complete Application Flow

1. User enters income, expenses, and financial goals.
2. Frontend JavaScript sends the data to `POST /analyse`.
3. Flask validates and sanitises the input.
4. Flask builds a structured financial prompt.

5. CATEGORY_TIPS and GOAL_DESCRIPTIONS provide additional guidance.
6. Flask sends the prompt to the Gemini API.
7. Gemini generates a financial analysis.
8. `extract_json()` extracts the structured JSON response.
9. Flask sends the structured result back to the frontend.
10. The frontend displays the budget, spending analysis, saving suggestions, and reports.

Personal Finance Advisor Bot — Architecture & AI Integration